



UNIVERSIDAD NACIONAL AUTÓNOMA DE MÉXICO

FACULTAD DE INGENIERÍA

Computación Gráfica e
Iteración Humano Computadora



Proyecto Final – Manual Técnico

PROFESOR:

Carlos Aldair Roman Balbuena

Nº DE CUENTA:

422113340

GRUPO: 5

SEMESTRE: 2026-1

ENTREGA: 15 / 11 / 2025



ÍNDICE

1. Objetivos	4
2. Alcance del proyecto	4
3. Limitantes.....	4
4. Diagrama de flujo del software	5
5. Diagrama de Gantt.....	6
6. Metodología de software aplicada	6
7. Tecnologías utilizadas e interacción del proyecto	7
7.1 Modelado 3D en Blender	7
7.2 Integración del proyecto: Visual Studio + OpenGL.....	8
7.3 Edición y generación de texturas	8
7.4 Interacción general del proyecto	9
8. Imágenes de referencia y modelado	9
9. Diccionario	20
9.1 Diccionario de Variables	20
9.2 Diccionario de Funciones	22
10. Documentación del código	22
10.1 Bibliotecas	22
10.2 Variables globales.....	23
10.3 Arreglo de vértices	23
10.4 Funciones de Modelado	24
10.4.1 Funciones para modelados de código en OpenGL.....	24
10.4.2 Funciones para Modelados OBJ	24
10.4.3 Función para cargar Texturas	25
10.5 Skybox.....	25
10.6 Modelados en código (OpenGL)	26
10.7 Modelados en Blender	27
10.8 Animaciones sencillas.....	27
10.8.1 cabeza animada	27
10.8.2 Sillas que se mueven	27
10.8.3 Puerta que se abre.....	27



10.9 Animaciones Complejas	27
7.9.1 Murciélagos volando	27
11. Análisis de Costos	28
Equipo de oficina:	28
12. Estado del arte del Proyecto	29
13. Conclusiones	30
14. Referencias	30



1. Objetivos

Estos son los objetivos planteados:

1. Desarrollar un recorrido virtual interactivo utilizando OpenGL que incluya la representación realista de una fachada y dos cuartos interiores, basados en imágenes de referencia seleccionadas cuidadosamente.
2. Integrar una cámara sintética manipulable en tiempo real, que permita la exploración libre y natural del espacio virtual, facilitando una experiencia inmersiva para el usuario.

2. Alcance del proyecto

El alcance de este proyecto consiste en diseñar y recrear una fachada arquitectónica utilizando técnicas de modelado 3D y animación. Para ello, se seleccionó un espacio ficticio: la **casa de Marceline**, perteneciente a la serie *Hora de Aventura*. La elección de esta construcción permite trabajar con libertad creativa.

El proyecto contempla la implementación de dos habitaciones y una porción del patio, enfocándose principalmente en la fachada como elemento central. La escena será construida mediante una combinación de:

- Modelos elaborados en OpenGL, generados a partir de primitivas geométricas.
- Modelos creados en Blender, exportados e integrados a la escena.

De igual forma, se desarrollarán cinco animaciones dentro del entorno: tres consideradas sencillas, basadas en transformaciones geométricas básicas (traslación, rotación y escala), y dos animaciones complejas, las cuales incluirán movimientos más avanzados, trayectorias no lineales o interacciones entre elementos. El resultado final será una escena dinámica, organizada y funcional, que represente adecuadamente el espacio elegido y demuestre el dominio de modelado, texturizado y animación en un entorno gráfico 3D programado en OpenGL.

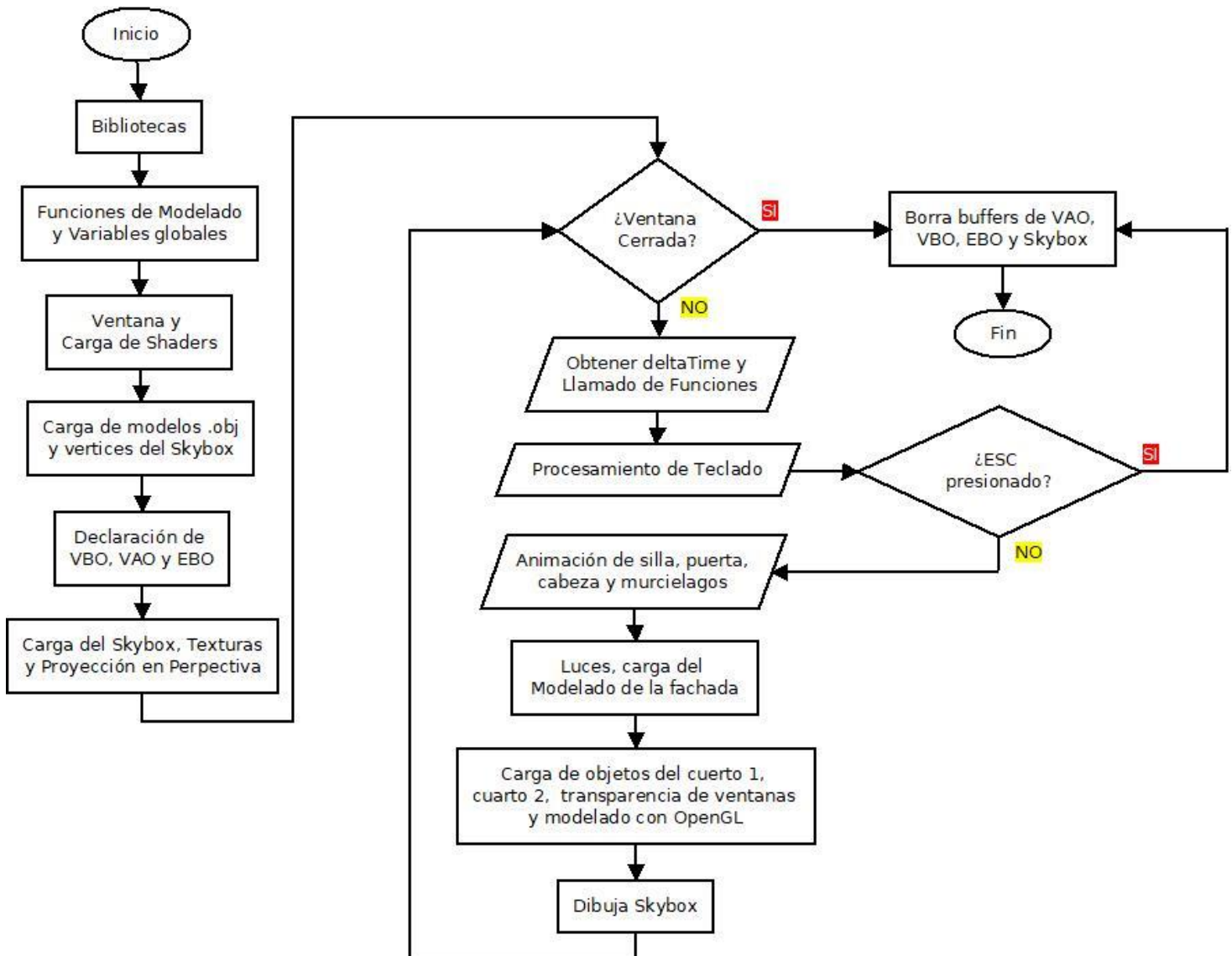
3. Limitantes

Para acotar el desarrollo y asegurar la viabilidad del proyecto dentro del tiempo estipulado, se establecieron las siguientes limitantes:

1. El modelado se restringe a un máximo de dos habitaciones, evitando expandir el escenario más allá de lo necesario para mantener el enfoque en la fachada.
2. El espacio recreado se limitará únicamente a la fachada y una parte del patio, sin incluir interiores complejos o estructuras adicionales de gran tamaño.
3. Todas las animaciones deberán ser programadas directamente en OpenGL, sin utilizar sistemas externos de animación o herramientas automáticas.
4. Se deberán modelar al menos siete objetos utilizando primitivas geométricas en OpenGL.
5. Se deberán integrar al menos siete objetos modelados previamente en Blender, respetando el flujo de exportación/importación hacia OpenGL.

6. Las animaciones complejas no podrán basarse únicamente en transformaciones geométricas simples ni en trayectorias lineales; deberán incluir comportamientos más elaborados, como curvas, combinaciones de movimientos o interacción entre objetos.
7. Las animaciones sencillas sí podrán estar basadas en transformaciones geométricas básicas y trayectorias lineales, ya que su objetivo es demostrar el dominio fundamental del pipeline de animación en OpenGL.

4. Diagrama de flujo del software



5. Diagrama de Gantt

	Días	Semana 1	Semana 2	Semana 3	Semana 4	Semana 5	Semana 6	Semana 7	Semana 8
Aprobación de Proyecto	1								
Aprobación de Objetos	1								
Modelado de 3 Objetos	3								
Modelado de 4 Objetos	3								
Avanzar Fachada	7								
Modelado de 3 Objetos	14								
Modelado Habitaciones	14								
Texturizado de 4 Objetos	14								
Implementación en OpenGL	14								
Texturizado de Fachada	14								
Texturizado de 6 Objetos	14								
Implementación de Skybox	7								
Corrección de errores código	7								
Texturizado de Habitaciones	7								
Corrección en Texturas	7								
Manual de Usuario	7								
Manual de Técnico	7								
Análisis de costos	7								
Completo									
Incompleto									

6. Metodología de software aplicada

Al iniciar el proyecto se optó por utilizar una **metodología tradicional en cascada**, debido a que su estructura lineal parecía adecuada para un proceso claramente definido:

1. Recolección de requisitos
2. Diseño del modelado
3. Texturizado
4. Implementación en OpenGL
5. Pruebas y correcciones
6. Documentación

Sin embargo, a medida que avanzó el proyecto, se identificó una limitación importante:

El modelo en cascada es rígido y no se adapta bien a los cambios una vez que una fase está formalmente cerrada.

Durante el desarrollo surgieron ajustes necesarios en modelos 3D, cambios en texturas, problemas de compatibilidad con OpenGL, reorganización de habitaciones y revisiones constantes de materiales. Esto hizo que el enfoque secuencial **frenara el progreso** y obligara a reabrir actividades que ya deberían estar cerradas según el modelo en cascada.

Ante esa problemática fue necesario adoptar un enfoque más flexible.

Se decidió adoptar **Scrum** como marco de trabajo ágil, ya que permite:

- Dividir el proyecto en **sprints** cortos.
- Revisar avances de forma continua.
- Entregar incrementos funcionales del producto.
- Ajustar prioridades sin afectar todo el plan completo.
- Integrar retroalimentación constante del profesor.

Scrum facilitó especialmente tareas como:

- Ajustes sobre modelos 3D
- Corrección de texturas que no funcionaban correctamente
- Animaciones que requerían iteraciones
- Pruebas en ejecución real dentro del motor de OpenGL
- Integración progresiva de luces, cámara, skybox y materiales

Esto permitió que el proyecto se mantuviera totalmente operativo mientras seguían agregándose funciones nuevas.

El proyecto terminó adoptando una **metodología híbrida**:

1. **Cascada** se utilizó para las fases generales y obligatorias establecidas desde el inicio (aprobación, modelado base, texturizado).
2. **Scrum** se aplicó dentro de cada gran fase mediante sprints, para iterar mejoras, corregir errores y ajustar el producto sin romper el flujo de trabajo.

Este enfoque híbrido permitió mantener el orden formal del cascada, pero con la flexibilidad del desarrollo ágil.

El diagrama de Gantt representa la planificación secuencial inicial del proyecto. A pesar de estar basado en cascada, cada bloque de varias semanas fue dividido internamente en **sprints de 7 o 14 días**, permitiendo iteraciones, revisiones y ajustes continuos.

7. Tecnologías utilizadas e interacción del proyecto

Para el desarrollo del proyecto fue necesario utilizar diversas herramientas de software que, en conjunto, permitieron modelar, texturizar, programar, compilar y visualizar todo el entorno 3D en tiempo real. A continuación, se describen las tecnologías empleadas y la forma en que interactúan entre sí dentro del flujo completo del proyecto.

7.1 Modelado 3D en Blender

Para la creación de todos los objetos y estructuras del proyecto se eligió **Blender**, principalmente por dos razones:

1. Es un **software gratuito**, lo que facilita trabajar sin restricciones de licencias.
2. Ofrece una gran libertad y control al momento de **modelar geometría**, permitiendo crear cada pieza del escenario con precisión.



Sin embargo, tal como se mencionó en clase, Blender tiene una **desventaja evidente en el manejo de texturas**, ya que su sistema de UV Mapping no resulta tan intuitivo como en otras herramientas profesionales como **Maya**, donde texturizar es un proceso más cómodo y visualmente sencillo. A lo largo del proyecto surgieron varias dificultades relacionadas con el estiramiento de texturas y la correcta asignación de materiales, problemas que no fueron tan presentes con Maya en las prácticas de laboratorio. No puedo hablar de primera mano sobre **3ds Max**, ya que no fue utilizado durante este proceso.

A pesar de ello, la eficiencia de Blender en el modelado y su accesibilidad económica lo convirtieron en la mejor opción para el proyecto.

7.2 Integración del proyecto: Visual Studio + OpenGL

Una vez terminados los modelos, estos fueron exportados e integrados en el proyecto mediante Visual Studio, el entorno principal de desarrollo. Este software resultó indispensable para:



- Escribir y organizar el código C++ del proyecto
- Administrar recursos, librerías externas y rutas de texturas
- Compilar y ejecutar el programa con soporte directo a las API de gráficos

Debido a la configuración del entorno, fue necesario trabajar sobre **Windows**, ya que Visual Studio ofrece una compatibilidad más directa con las librerías utilizadas. No estoy completamente seguro de si toda la configuración podría replicarse fácilmente en Linux, aunque es posible que se requiera otra cadena de herramientas distinta.



El proyecto utiliza la API **OpenGL**, junto con librerías como **GLFW**, **GLAD**, **SOIL/STB_Image** y **ASSIMP**, para manejar ventanas, cargar modelos, inicializar shaders y procesar texturas. No se documenta cada librería en detalle porque todas ellas cuentan con documentación oficial amplia y suficiente en línea.

7.3 Edición y generación de texturas

Otro componente fundamental del proyecto fue la **creación y edición de imágenes** para usarlas como texturas en los modelos 3D. Para ello se emplearon diferentes herramientas según la necesidad:

- **Desde el celular:** PicsArt y Sketchbook
- **Desde la computadora:** Microsoft Paint
- **Herramientas de IA:** ChatGPT (generación de texturas personalizadas) y Gemini (obtención y limpieza de imágenes de referencia)



El uso combinado de editores tradicionales e inteligencia artificial permitió obtener texturas más detalladas, corregir errores visuales y generar imágenes muy similares a las referencias originales del proyecto.

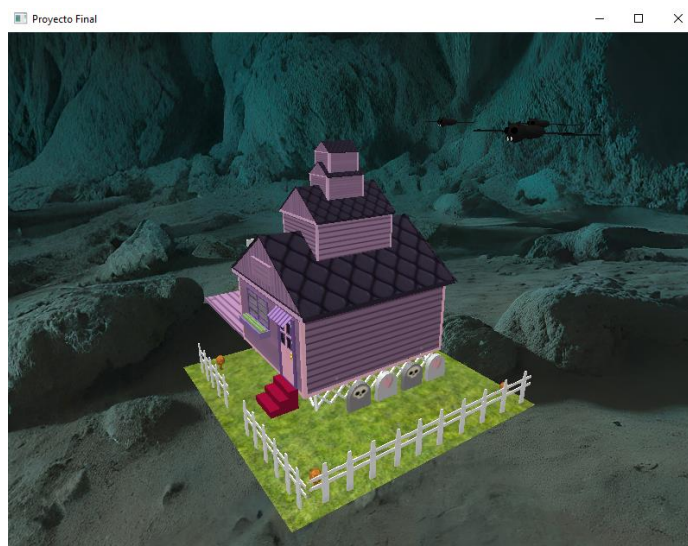
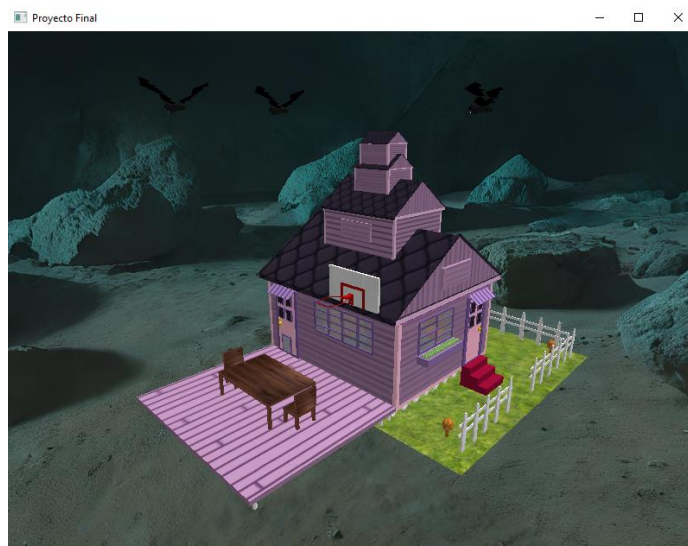
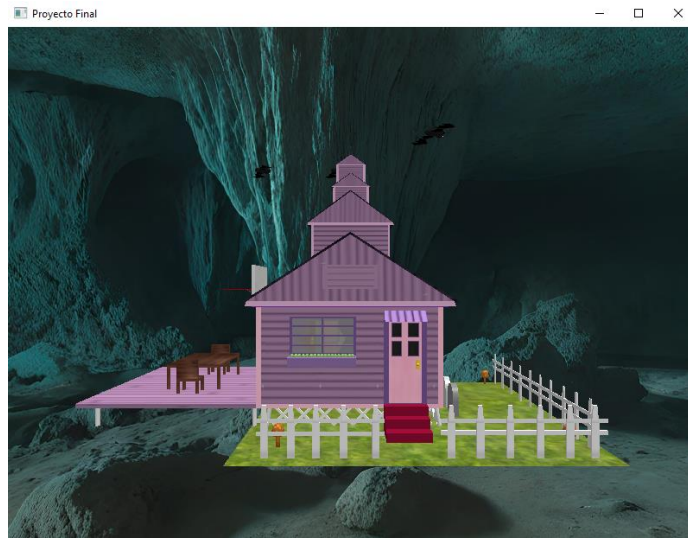
7.4 Interacción general del proyecto

La interacción entre todas las tecnologías puede resumirse en los siguientes puntos:

1. **Modelado** en Blender
2. **Exportación del modelo** (OBJ/FBX/GLTF)
3. **Edición o creación de texturas** en PicsArt, Sketchbook, Paint o IA
4. **Importación** de modelos y texturas hacia Visual Studio
5. **Implementación en C++** para animaciones, luces, cámara, físicas y lógica del proyecto
6. **Procesamiento mediante OpenGL**, shaders y matrices de transformación
7. **Ejecución final del entorno 3D interactivo**

8. Imágenes de referencia y modelado



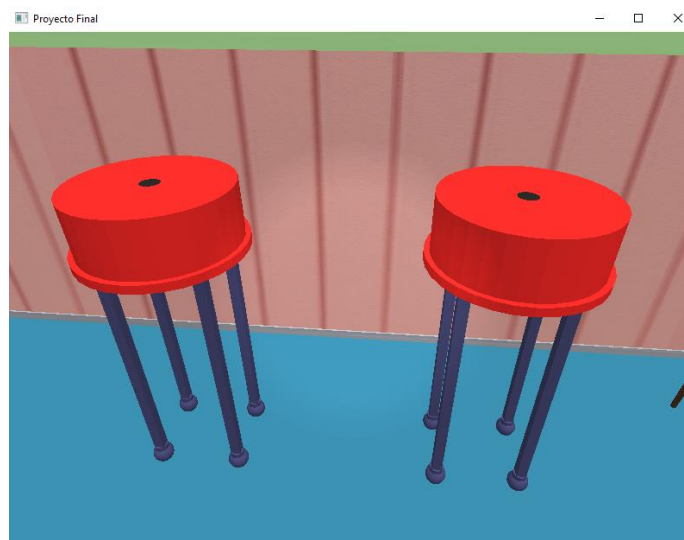


HABITACION 1

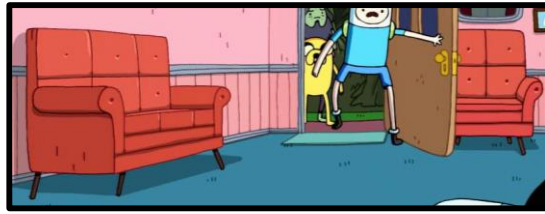


Lista de objetos de la habitación 1

1. Bancos de cocina:



2. Sofás:



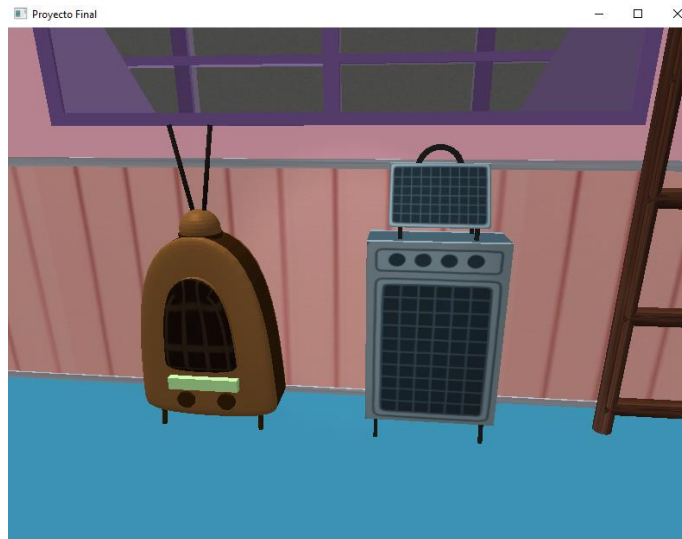
3. Amplificador:



4. Lámpara

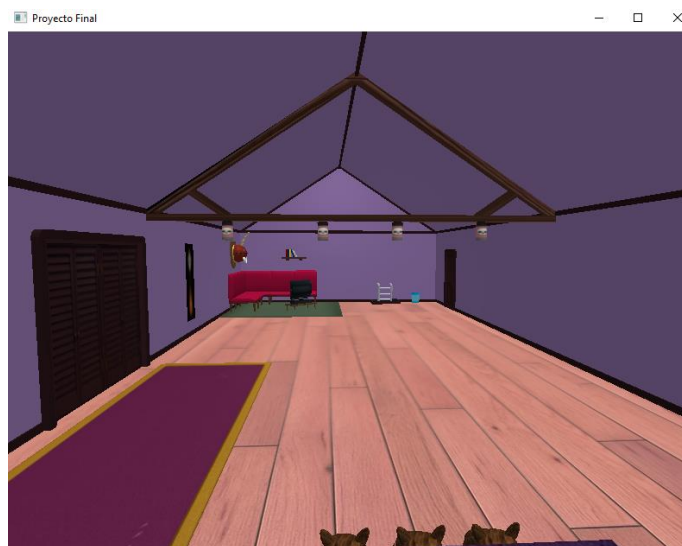


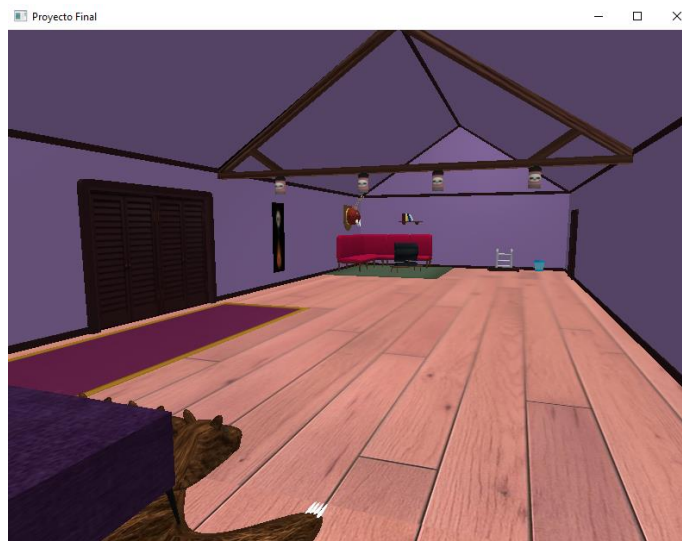
5. Radio



HABITACION 2



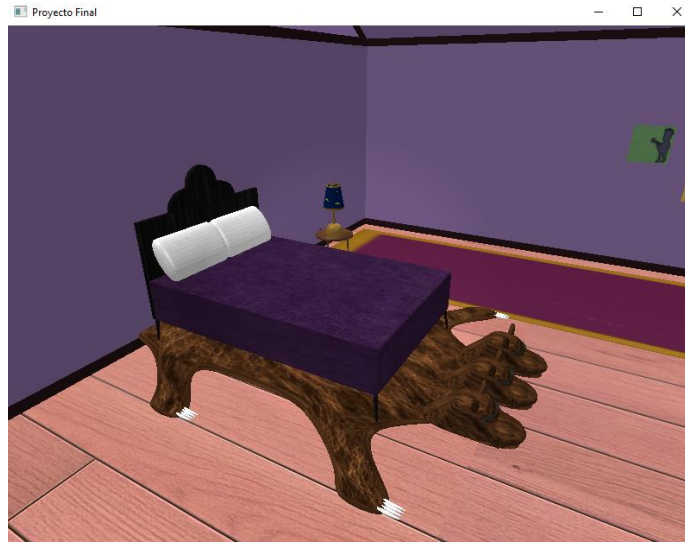




Lista de objetos de la habitación 2:

1. Cama:





2. Sofá:



3. Televisión:



4. Lámpara





5. Cabeza decorativa:





9. Diccionario

9.1 Diccionario de Variables

Variables Globales	Tipo	Descripción
WIDTH, HEIGHT	const GLuint	Resolución base de la ventana OpenGL (800×600).
SCREEN_WIDTH, SCREEN_HEIGHT	int	Resolución real del framebuffer de la ventana.
Camera	Camera	Objeto cámara principal (controla vista, posición y orientación).
lastX, lastY	GLfloat	Última posición del mouse usada para calcular offsets.
keys[1024]	Bool	Arreglo con el estado de todas las teclas (presionadas/soltadas).
firstMouse	Bool	Evita saltos bruscos en el primer movimiento del mouse.
lightPos	glm::vec3	Posición inicial de una luz puntual.
Active	Bool	Activa o desactiva la luz animada.
pointLightPositions[]	glm::vec3	Arreglo con posiciones de luces puntuales.
vertices[]	float	Arreglo con vértices de modelos dibujados con OpenGL.
indices[]	GLuint	Arreglo de índices de dibujo para modelos con EBO.
Light1	glm::vec3	Color base de la luz animada que cambia con el tiempo.
MAX_FRAMES	#define (int)	Máximo de frames usados para animaciones.
i_max_steps	int	Número máximo de pasos en animaciones progresivas.
i_curr_steps	int	Contador de pasos en una animación.
desplazamientoSilla	float	Posición actual de desplazamiento de una silla animada.
direccionSilla	int	Dirección de movimiento de la silla (1 = +X, -1 = -X, 0 = quieto).

Variables Globales	Tipo	Descripción
deltaTime	GLfloat	Tiempo entre el frame actual y el anterior.
lastFrame	GLfloat	Tiempo del frame anterior.
penduloAngulo	float	Ángulo actual del péndulo.
penduloVelocidad	Float	Velocidad de oscilación del péndulo.
penduloAmplitud	Float	Amplitud máxima en grados del péndulo.
bodyY	Float	Posición vertical del cuerpo del murciélago.
wingRot	Float	Rotación actual de las alas.



headRot	Float	Rotación actual de la cabeza.
up	bool	Bandera para movimiento vertical (subiendo/bajando).
wingUp	bool	Bandera para animación de alas (abriendo/cerrando).
bodyX	Float	Posición en X del movimiento elíptico del murciélago.
bodyZ	Float	Posición en Z del movimiento elíptico del murciélago.
bodyRotY	Float	Rotación del murciélago según dirección de vuelo.
batState	int	Máquina de estados (0 = idle, 1 = volando).
timeElapsed	Float	Tiempo acumulado usado en la animación del murciélago.
ellipseA	Float	Radio mayor de la elipse de vuelo (eje X).
ellipseB	Float	Radio menor de la elipse de vuelo (eje Z).
anguloPuerta	float	Ángulo actual de apertura/cierre de la puerta.
velocidadPuerta	Float	Velocidad de abertura de la puerta (grados/segundo).
abrirPuerta	Bool	Flag para iniciar animación de apertura.
cerrarPuerta	Bool	Flag para iniciar animación de cierre.
tiempoTotal	Float	Tiempo global para animaciones suaves (oscilaciones).
escalaModelo1	Float	Escala del modelo 1 (ejemplo: mesa).

Estas son las variables utilizadas en la función principal:

Variable de la Función main()	Tipo	Función
Window	GLFWwindow*	Control de la ventana
Projection	glm::mat4	Matriz de proyección
VBO, VAO, EBO	GLuint	Buffers para modelo principal
skyboxVBO, skyboxVAO	GLuint	Buffers del skybox
skyboxVertices[]	GLfloat[]	Vértices del skybox
faces	vector<char*>	Texturas del skybox
cubemapTexture	GLuint	Textura cúbica del skybox
lightingShader, lampShader, skyboxshader	Shader	Programas shader
Model ...	Model	Objetos del mundo 3D
texture1..14	GLuint	Texturas cargadas
Velocidad	float	Velocidad animación silla
model, modelTemp, modelTemp1	glm::mat4	Transformaciones
lightColor	glm::vec3	Color dinámico de luz
modelLoc, viewLoc, projLoc	GLint	Ubicaciones uniformes
currentFrame	float	Tiempo actual
deltaTime	float	Tiempo entre frames

9.2 Diccionario de Funciones

Nombre de la Función	Tipo	Descripción
KeyCallback()	void	Maneja todas las entradas del teclado. Detecta teclas presionadas/soltadas y activa funciones como movimiento de cámara, luces, animación de sillas y apertura/cierre de la puerta.
MouseCallback()	void	Procesa el movimiento del mouse para controlar la orientación de la cámara en primera persona. Actualiza desplazamientos en X y Y.
DoMovement()	void	Aplica el movimiento continuo de la cámara según teclas W, A, S, D o flechas. También permite rotar la cámara con Q y E.
Animation()	void	Controla animaciones automáticas como el movimiento elíptico del personaje, vibración del cuerpo, rotación de alas y oscilación de la cabeza.
Dibujar()	glm::mat4	Dibuja un cubo básico aplicando traslación y escala . Útil para objetos sin rotación. Retorna el modelo original previo.
DibujarR()	glm::mat4	Dibuja un cubo con rotación , escala y traslado . Ideal para partes móviles como brazos, hélices, mecanismos, etc.
DibujarC()	glm::mat4	Dibuja una caja personalizada permitiendo rotación, escala, traslado y elegir cuántos vértices dibujar (según "cara").
DibujarMu()	glm::mat4	Dibuja un modelo 3D completo con rotación, escala y traslación. No dibuja vértices directamente, solo envía la transformación al shader.
DibujarM()	glm::mat4	Variante más simple de DibujarMu(), también aplica traslación, rotación y escala, devolviendo el modelo transformado.
DibujarMu2()	glm::mat4	Variante de transformaciones para modelos, con orden de transformaciones diferente. Devuelve la matriz final en lugar de la original.
loadTexture()	GLuint	Carga una textura desde archivo usando stb_image. Configura filtrado, wrapping, blending y genera mipmaps. Retorna el ID de la textura.

10. Documentación del código

10.1 Bibliotecas

Estas serán las bibliotecas que nos ayudarán a lograr recrear la escena 3D de nuestro proyecto, ya sea para texturizar, modelar y moverte en la pantalla:

```
#include <iostream>
#include <cmath>
// GLEW
#include <GL/glew.h>
// GLFW
#include <GLFW/glfw3.h>
// Other Libs
#include "stb_image.h"
// GLM Mathematics
#include <glm/glm.hpp>
#include <glm/gtc/matrix_transform.hpp>
#include <glm/gtc/type_ptr.hpp>
//Load Models
#include "SOIL2/SOIL2.h"
// Other includes
#include "Shader.h"
#include "Camera.h"
#include "Model.h"
#include "Texture.h"
```

Son las mismas bibliotecas que trabajamos a lo largo del curso. Lo único que agregue fue la biblioteca de Texture.h que me servirá para implementar el Skybox.

10.2 Variables globales

Empecemos a definir las variables globales más importantes que me ayudaran en el modelado.

Yo considero estas variables muy importantes porque me ayudan a mí a escalar los modelos hechos con código desde OpenGL, y de esta forma amueblar la casa, me resulta más fácil, y puede definir el tamaño adecuado:

```
//escalas de modelos de OpenGL
float escalaModelo1 = 0.5f; // aumentar o disminuir la mesa
float escalaModelo2 = 0.5f;
float escalaModelo3 = 0.25f;
float escalaModelo4 = 0.25f;
float escalaModelo5 = 0.5f;
float escalaModelo6 = 0.25f;
float escalaModelo7 = 0.25f;
float escalaModelo8 = 0.3f;
```

También tengo variables de animación, por ejemplo, las del murciélago:

```
// Variables de animación, murciélago
float bodyY = 0.0f;
float wingRot = 0.0f;
float headRot = 0.0f;
bool up = true;
bool wingUp = true;
float bodyX = 0.0f;
float bodyZ = 0.0f;
float bodyRotY = 0.0f;
// Máquina de estados
int batState = 0; // 0: idle, 1: volando
float timeElapsed = 0.0f;
float ellipseA = 2.5f; // Radio en eje X
float ellipseB = 1.5f; // Radio en eje Z
```

10.3 Arreglo de vértices

En el código tendremos dos arreglos de vértices, uno para los modelados con transformaciones geométricas:

```
//Modelado de Objetos
float vertices[] = {
    -0.5f, -0.5f, -0.5f, 1.0f, 1.0f, 1.0f, 0.0f,0.0f, //frente
```

Y otra para el Skybox:

```
//Skybox
GLfloat skyboxVertices[] = {
    // Positions
    -1.0f, 1.0f, -1.0f,
```



Es importante mencionar en este punto que el EBO será configurado, o asignado a los índices de los vértices de los modelador, porque tuve problemas por la mala configuración:

```
glBindBuffer(GL_ELEMENT_ARRAY_BUFFER, EBO);  
glBufferData(GL_ELEMENT_ARRAY_BUFFER, sizeof(indices), indices,  
GL_STATIC_DRAW);
```

10.4 Funciones de Modelado

Yo decidí implementar funciones para implementar transformaciones geométricas y reducir el número de líneas de código que se repetirían en la función main(). Por lo tanto, Con estas funciones hace que sea más sencillo Aplicar transformaciones y poder aplicar el modelado jerárquico de tal forma que se fácil de entender para la persona que lea este documento y trabaje con el proyecto.

10.4.1 Funciones para modelados de código en OpenGL

Esta es una de las primeras funciones que programe, donde solo escala y traslada. Conforme fui modelando más objetos, me vi en la necesidad de volver a hacer más funciones, pero con parámetros de rotación y numero de triángulos.

```
glm::mat4 Dibujar(glm::mat4 model, glm::vec3 escala, glm::vec3 traslado,  
GLint uniformModel) {  
    glm::mat4 modelTemp = model; // copia la transformación actual  
    modelTemp = glm::translate(modelTemp, traslado);  
    model = glm::scale(model, escala);  
    glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));  
    glDrawArrays(GL_TRIANGLES, 0, 36);  
  
    return modelTemp; // devolvemos la transformación  
}
```

10.4.2 Funciones para Modelados OBJ

Es el mismo análisis que la función para transformar modelos de código en OpenGL, solo que esta se encarga de escalas, trasladar y rotar los modelos que hice en blender.

```
glm::mat4 DibujarM(glm::mat4 model, glm::vec3 traslado, glm::vec3 escala,  
glm::vec3 rotacion, float angulo, GLint modelLoc) {  
    glm::mat4 modelTemp = model; // copia la transformación actual  
    modelTemp = glm::translate(modelTemp, traslado);  
    model = glm::rotate(model, glm::radians(angulo), rotacion);  
    model = glm::scale(model, escala);  
  
    glUniformMatrix4fv(modelLoc, 1, GL_FALSE, glm::value_ptr(model));  
  
    return modelTemp; // devolvemos la transformación  
}
```

10.4.3 Función para cargar Texturas

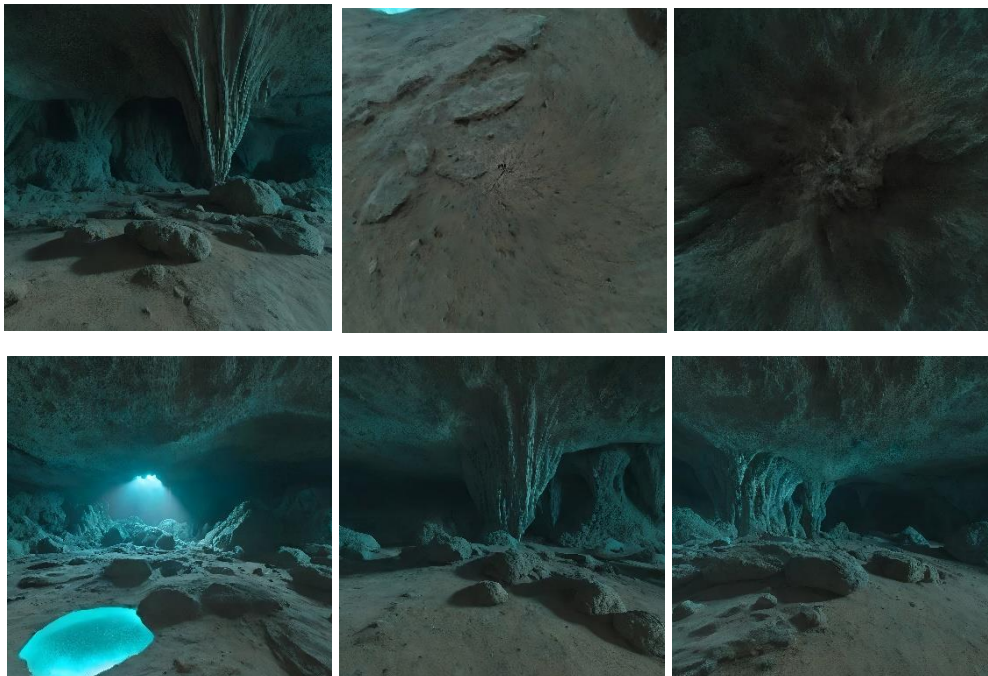
Esta función es a mi parecer la más importante porque me permite generar ID de texturas para después ponérselas a mis modelos de código en OpenGL. (La verdad fue una de las partes donde más me costó porque no tenía ni idea de cómo implementar más de una textura hasta ese momento)

```
GLuint loadTexture(const char* path) {  
    glEnable(GL_BLEND);  
    glBlendFunc(GL_SRC_ALPHA, GL_ONE_MINUS_SRC_ALPHA);  
    ...  
}
```

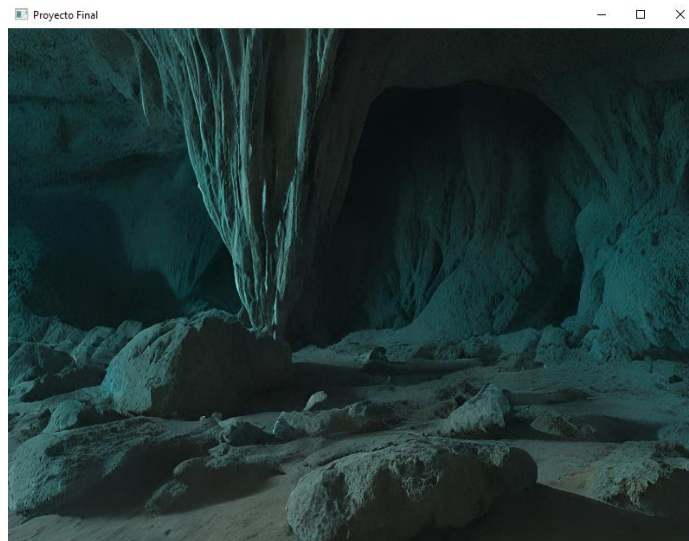
10.5 Skybox

Para diseñar el Skybox de mi proyecto, no fue una tarea sencilla, se requirió de los archivos del siguiente canal de Youtube, donde poder llenar un formulario para que te hagan llegar los archivos necesarios para su implementación.

Gracias a seguir los pasos del video fue que pude implementar mi skybox, para dividirla en 6 caras entre a este enlace publicado en un foro de Reddit



Así se ve ya implementada:



Explicando la parte más importante es diseñar una propia VAO para el Skybox, porque lo que se me dificultó bastante fue que me cargaba el Skybox, pero no las texturas que hice en código, pero me di cuenta que el problema fue que declare dos veces la EBO, uno para cada arreglo de vértices, pero si solo definía la EBO para los vértices del modelado en código, eso fue lo que me funcionó.

En resumen, los errores que me enfrenté fueron los siguientes:

- Si usas el mismo VAO que tus modelos, el skybox se dibuja mal o no se ve
- Si habilitas atributos extra (normales/UV), se corrompen las texturas
- Si no quitas la traslación de la vista parece que el skybox se mueve contigo
- Si cargas el cubemap usando un VAO equivocado, pierdes texturas de modelos o del skybox

10.6 Modelados en código (OpenGL)

Se hace uso de las funciones ya antes mencionadas, solo consiste en asignar parámetros de donde quieres que se posicione, se oriente y su tamaño, en cuanto a los modelos que hice fue

1. Una mesa con dos sillas
2. Un refrigerador
3. Un mueble con horno de microondas
4. Un librero
5. Un cuadro con retrato incluido
6. Una caja de cartón con cintas VHS
7. Un reloj con péndulo



10.7 Modelados en Blender

De igual forma, se hace uso de las funciones que implementar para posicionar, rotar y escalar el tamaño de los objetos exportados desde blender. Los objetos que yo hice fueron los siguientes

1. Una tumba
2. Un hongo
3. Dos repisas con veladoras
4. Un bote de basura
5. Un murciélago
6. Una mesa con una especie de radioemisora
7. Objeto, tipo piedra de cueva, que caerá hasta el suelo

10.8 Animaciones sencillas

10.8.1 cabeza animada

Solo se aplica una transformación de rotación para simular el movimiento que se espera.

10.8.2 Sillas que se mueven

Simulan como si una persona se fuera a sentar, y tiene que arrastrar las sillas. En este Solo se hace una traslación.

10.8.3 Puerta que se abre

Mi última animación es una puerta que se abre del mueble, y se puede llegar a cerrar de nuevo.

10.9 Animaciones Complejas

7.9.1 Murciélagos volando

La animación del murciélago en el proyecto se basa en funciones trigonométricas para simular tres aspectos principales:

1. El aleteo (alas)
2. El movimiento del cuerpo (subida/bajada)
3. La trayectoria de vuelo (rotación y desplazamiento)

Estas funciones se usan porque generan movimientos suaves, periódicos y naturales, tal como ocurre en los animales reales.



11. Análisis de Costos

Teniendo en cuenta el desarrollo del proyecto de tres meses se obtiene el siguiente análisis de costos.

Mano de obra

- **Desarrollador:** Para este proyecto se contó con un solo desarrollador, responsable del modelado, texturizado, programación y documentación. Se consideró una percepción mensual neta de **\$12,500.00 MXN**.

Herramientas y Software

- **Licencias:** Para el desarrollo del proyecto fue necesario contar con una licencia de software de diseño 3D durante **2 meses**, con un costo mensual de **\$2,800 MXN**.

Gastos indirectos

- **Energía eléctrica:** Para la ejecución del proyecto, el precio mensual es de **\$350 MXN**.
- **Internet:** El precio del servicio mensual es de **\$450 MXN**.

Depreciación de activos

Equipo de cómputo:

- **Costo original:** \$28,000 MXN
- **Tasa de depreciación fiscal anual:** 30%

$$\text{Depreciación mensual} = \frac{(28,000)(0.30)}{12} = \$700 \text{ MXN}$$

Equipo de oficina:

- **Costo original:** \$3,200 MXN
- **Tasa de depreciación fiscal anual:** 10%

$$\text{Depreciación mensual} = \frac{(3,200)(0.10)}{12} = \$26.66 \text{ MXN}$$

CATEGORÍA	COSTO (MXN)
MANO DE OBRA (3 MESES)	\$37,500.00
LICENCIA DE SOFTWARE (2 MESES)	\$5,600.00
ENERGÍA ELÉCTRICA E INTERNET (3 MESES)	\$2,400.00
DEPRECIACIÓN EQUIPO DE CÓMPUTO (3 MESES)	\$2,100.00
DEPRECIACIÓN EQUIPO DE OFICINA (3 MESES)	\$79.98
TOTAL ESTIMADO SIN UTILIDAD	\$47,679.98
CONTINGENCIA (10%)	\$4,767.99
SUBTOTAL	\$52,447.97
UTILIDAD (12%)	\$6,293.75
SUBTOTAL CON UTILIDAD	\$58,741.72
IVA (16%)	\$9,398.67
TOTAL CON IVA	\$68,140.39

12. Estado del arte del Proyecto

Los recorridos virtuales se han convertido en una herramienta fundamental en diversas áreas como el entretenimiento, la arquitectura, la educación y los bienes raíces. Su popularidad ha aumentado debido al avance de tecnologías gráficas, motores 3D, dispositivos de realidad virtual y técnicas optimizadas para representar espacios digitales de manera cada vez más inmersiva y realista. Actualmente, los recorridos virtuales permiten a los usuarios explorar entornos tridimensionales sin limitaciones físicas, facilitando la visualización de espacios inexistentes, difíciles de visitar o completamente ficticios.

En la industria del entretenimiento y los videojuegos, los recorridos virtuales se utilizan para crear mundos interactivos que brindan experiencias dinámicas y atractivas. Las series animadas, películas y franquicias icónicas suelen ser representadas en entornos digitales creados por la comunidad, permitiendo a los usuarios explorar lugares ficticios con un alto grado de libertad visual y estética. En el ámbito arquitectónico, los recorridos virtuales se han convertido en una herramienta estándar para presentar proyectos antes de su construcción, optimizando la toma de decisiones y mejorando la comunicación entre clientes y diseñadores. Asimismo, en educación, se emplean para recrear escenarios históricos o científicos que enriquecen los procesos de enseñanza-aprendizaje.

En este contexto, el desarrollo de un recorrido virtual de la casa de Marceline, personaje de la serie *Hora de Aventura*, se alinea con las tendencias actuales de recreación digital de espacios ficticios. En este proyecto se modeló la estructura principal de la casa junto con diez objetos adicionales —entre ellos cama, lámparas y elementos decorativos— tomando como referencia imágenes originales para lograr colores y proporciones fieles. Además, se integraron cuatro animaciones que añaden dinamismo al escenario, siguiendo las prácticas modernas de diseño interactivo en entornos 3D.

El uso de herramientas de modelado, texturizado y animación, junto con técnicas de iluminación y renderizado, permiten que este recorrido virtual mantenga coherencia estética con la obra original, al tiempo que aprovecha los avances actuales en visualización tridimensional. Este tipo de proyectos muestra cómo los recorridos virtuales no solo replican espacios reales, sino que también funcionan como medio creativo para interpretar mundos imaginarios con un alto grado de detalle e inmersión.

13. Conclusiones

Con el desarrollo del proyecto, la parte del modelado y las animaciones resultó ser la más entretenida y enriquecedora. Implementar transformaciones, trayectorias y movimientos complejos me permitió aplicar conceptos fundamentales de gráficos computacionales y ver resultados visuales inmediatos. Además, la creación del skybox y la ambientación general del entorno me ayudaron a comprender cómo pequeños elementos pueden mejorar significativamente la percepción del escenario.

Este proyecto también me mostró el impacto que tiene la computación gráfica en la experiencia del usuario. Detalles como la forma del modelo, la iluminación o la calidad del texturizado pueden provocar aceptación o rechazo en quien observa la escena. Por ello, me esforcé en cuidar los detalles de las fachadas y en generar un entorno visual coherente, buscando transmitir una buena impresión desde el primer momento.

En términos generales, este trabajo fortaleció mi comprensión de que la computación gráfica no solo consiste en dibujar objetos en pantalla, sino en tomar decisiones de diseño que influyen directamente en cómo las personas perciben e interpretan un espacio. Cada elección técnica o estética tiene un impacto, y dominar estos procesos permite crear experiencias más sólidas, atractivas y significativas.

14. Referencias

- Crane, L. (s. f.). *Panorama to Cubemap* [Herramienta web]. Recuperado de <https://jaxry.github.io/panorama-to-cubemap/> Jaxry
- Implementación de un Skybox en OpenGL: Cielo y Ambiente Realista para Escenas 3D [Video]. (2025, ...) YouTube. Recuperado de <https://youtu.be/VDfRE9VjeAA?si=pPld2EmGDU3kRFeY>